

基于 openGauss 的学生选课成绩管理系统 数据库设计与实现分析

数据库原理 2 大作业期末报告

课程名称：数据库原理 2

项目名称：学生选课成绩管理系统

数据库平台：openGauss

应用技术：Python、Streamlit、psycopg2、Docker

学生姓名：请填写姓名

学号：请填写学号

专业班级：请填写专业班级

完成日期：2026 年 6 月 8 日

本报告依据当前项目仓库真实 SQL、服务层代码和运行脚本撰写

摘要

本文以一个面向高校教务场景的学生选课成绩管理系统为研究对象，按照数据库系统设计流程，完成从规划、需求分析、概念结构设计、逻辑结构设计、规范化设计、完整性约束、事务并发控制、物理设计到运行维护的数据库设计报告。项目当前使用 Docker 运行 openGauss，应用层采用 Python、Streamlit 和 psycpg2，通过原生 SQL 访问数据库。

报告重点不是介绍页面功能，而是围绕数据库原理课程内容分析系统的数据模型。系统将课程定义与开课班次分离，使用 enrollment 将学生与教学班之间的多对多联系转换为关系模式；通过 course_schedule 将上课时间片结构化，删除旧设计中可能破坏 1NF 或 3NF 的 schedule_text、selected_count 和 gpa_point 等派生字段；通过主键、外键、唯一约束、CHECK 约束、视图和触发器维护实体完整性、参照完整性和用户自定义完整性。当前数据库层已经实现角色档案一致性、先修课程合法性、选课完整性、教室容量一致性和成绩变更审计触发器；在选课流程中使用 select_course_tx()、DBSession 和 SELECT...FORUPDATE 锁定目标开课班次，以降低并发抢课导致超选的风险；并通过 test_triggers_constraints.sql 对关键规则进行回滚型验证。

通过该项目可以较完整地体现数据库设计流程、ER 建模、关系模型转换、范式分析、完整性约束、事务 ACID、锁机制、索引设计和运行维护等数据库原理 2 课程知识点。

关键词：openGauss；选课系统；ER 模型；关系模型；3NF；完整性约束；事务；行级锁

目录

摘要	1
第 1 章 绪论与系统规划	5
1.1 项目背景	5
1.2 系统规划目标	5
1.3 数据库设计流程说明	5
1.4 本章课程知识点	6
第 2 章 需求分析	7
2.1 用户角色	7
2.2 功能需求	7
2.3 数据需求	8
2.4 业务规则	8
2.5 数据字典草案	9
第 3 章 概念结构设计	10
3.1 实体抽取	10
3.2 属性设计	10
3.3 联系与基数	10
3.4 ER 图	11
3.5 从业务语义到概念模型	11
第 4 章 逻辑结构设计	12
4.1 关系模式	12
4.2 关系模式、主键、外键与唯一约束	12
4.3 多对多关系转换	13
4.4 课程定义与开课班次分离	14
4.5 多值属性与自关联关系处理	14
4.6 逻辑关系模型图	14
4.7 参照完整性	14
第 5 章 规范化设计与范式分析	16
5.1 规范化分析对象与方法	16

5.2	当前关系模式与主键、候选键、外键	16
5.3	核心函数依赖分析	17
5.4	第一范式分析：属性原子性与结构化时间片	18
5.5	第二范式分析：复合键与部分函数依赖	19
5.6	第三范式分析：传递依赖、派生字段与表分解	19
5.7	BCNF 分析：决定因素与候选键	20
5.8	从旧设计到 3NF 的迁移说明	20
5.9	当前设计中仍可改进的规范化问题	21
5.10	本章小结	22
第 6 章	完整性约束、触发器与检查机制	23
6.1	完整性约束设计目标	23
6.2	实体完整性：主键约束	23
6.3	参照完整性：外键约束	23
6.4	用户自定义完整性：UNIQUE、CHECK、NOT NULL、DEFAULT	23
6.5	跨表业务完整性与触发器设计	24
6.6	选课完整性触发器	25
6.7	成绩变更审计触发器	25
6.8	角色档案一致性触发器	26
6.9	先修课程合法性触发器	26
6.10	应用层校验与数据库触发器的关系	26
6.11	本章小结	26
第 7 章	事务管理、锁机制与并发控制	27
7.1	选课系统中的并发一致性问题	27
7.2	选课事务边界设计	27
7.3	选课事务流程	27
7.4	行级锁防止并发超选	28
7.5	唯一约束与重复选课防护	28
7.6	时间冲突与先修课程检查的事务一致性	29
7.7	成绩更新事务与审计一致性	29
7.8	事务隔离级别分析	29
7.9	当前实现的不足与改进	30
7.10	本章小结	30
第 8 章	物理结构设计与性能优化	31
8.1	数据库系统选择	31

8.2	数据类型设计	31
8.3	索引设计	31
8.4	规范化后的索引补偿	32
8.5	视图设计与派生数据查询	32
8.6	高频查询	33
8.7	性能风险与优化	33
8.8	备份恢复	33
第 9 章	系统运行、维护与数据库管理	34
9.1	数据库初始化与部署	34
9.2	迁移脚本与部署顺序	34
9.3	测试脚本与验证	34
9.4	权限管理	34
9.5	日志审计	35
9.6	异常处理与一致性维护	35
9.7	视图与查询维护	35
9.8	数据安全	35
9.9	后续扩展	35
第 10 章	总结与不足	36
10.1	系统设计总结	36
10.2	课程知识点体现	36
10.3	项目不足	36
10.4	后续改进	37
附录 A	关键 SQL 与测试摘录	38
A.1	选课事务函数摘录	38
A.2	角色一致性触发器摘录	38
A.3	选课完整性触发器摘录	39
A.4	成绩审计触发器摘录	39
A.5	视图 SQL 摘录	40
A.6	SQL 测试结果摘录	40
参考资料		41

第 1 章 绪论与系统规划

1.1 项目背景

本项目是一个面向高校教务场景的学生选课成绩管理系统。系统需要支撑学生查询课程、提交选课和退课申请，教师录入成绩并查看成绩分布，管理员维护学期、课程、教学班、学生和教师基础资料。与普通信息展示系统相比，选课系统的核心难点在于数据联系复杂、完整性约束多、并发选课场景明显。课程容量、重复选课、上课时间冲突、先修课程、成绩录入和退课限制都属于典型数据库规则。

项目当前采用 Python + Streamlit 实现界面，使用 psycopg2 连接 openGauss 数据库。数据库部署相关文件位于 `opengauss_setup/docker/`，核心建表脚本位于 `opengauss_setup/sql/init.sql`。项目没有使用 ORM，也没有使用 Alembic 等迁移框架，数据库模式主要由 SQL 文件维护。

1.2 系统规划目标

根据数据库设计流程，规划阶段首先需要明确系统边界和数据管理目标。本系统的规划目标包括：

1. 建立统一的用户账号与角色体系，支持学生、教师、管理员三类用户。
2. 区分课程定义和具体教学班，支持同一课程在不同学期、不同教师、不同教室开设。
3. 使用选课记录表达学生和教学班之间的多对多联系，并保存选课状态、选课时间和最终成绩。
4. 通过结构化上课时间支持时间冲突检查。
5. 通过主键、外键、唯一约束、CHECK 约束和服务层校验共同保证数据完整性。
6. 在并发选课场景中使用事务和锁机制降低超选风险。

1.3 数据库设计流程说明

本报告按照数据库设计流程组织章节：需求分析识别角色、功能、数据和规则；概念设计使用 ER 模型抽象实体和联系；逻辑设计把 ER 模型转换为关系模式；规范化设计分析函数依赖和范式；完整性设计讨论主键、外键、CHECK、UNIQUE 和应用层校验；事务与锁章节分析并发选课一致性；物理设计讨论数据类型、索引和性能；运行维护章节说明初始化、部署、安全和扩展。

1.4 本章课程知识点

本章对应数据库系统设计流程中的规划阶段。规划阶段强调数据库系统建设的必要性、系统边界、软硬件环境、数据范围和设计目标。本项目的数据中心特征明显：无论学生选课、教师录入成绩还是管理员维护教学班，本质上都是围绕数据库中实体、联系和约束展开的操作。

第2章 需求分析

2.1 用户角色

项目通过 `user_account.role` 区分 `admin`、`student`、`teacher` 三类角色。登录认证由 `services/auth_service.py` 实现，页面访问控制由 `pages/_guards.py` 中的 `require_role()` 完成。三类用户需求如下：

- 学生：查看当前学期可选课程，提交选课，查看已选课程，按规则退课，查看成绩单和绩点。
- 教师：查看本人负责的开课班次，录入或修改学生成绩，查看成绩分布和成绩修改日志。
- 管理员：维护学期、课程、开课班次、学生、教师，查看系统统计和所有班次成绩。

2.2 功能需求

系统功能需求和真实代码对应关系见表 2-1。

表 2-1 功能需求与项目文件对应关系

功能	当前实现	主要文件
登录与会话	用户名密码登录，Cookie 持久会话，按角色进入不同页面	<code>services/auth_service.py</code> 、 <code>utils/auth_cookie.py</code> 、 <code>opengauss_setup/sql/add_user_session.sql</code>
课程查询	学生、教师、管理员均可通过服务层查询课程与开课班次	<code>services/course_service.py</code>
选课	检查重复、容量、学期状态、选课窗口、先修课、时间冲突后写入 <code>enrollment</code>	<code>services/selection_service.py</code>
退课	只能退本人已选且未录入成绩的课程，并检查学期和时间窗口	<code>services/selection_service.py</code>
课程与教学班管理	管理员维护课程、学期、开课班次和上课时间片	<code>services/course_service.py</code> 、 <code>pages/course_manage.py</code>
成绩管理	教师只能维护本人班次，管理员可维护全部班次，并记录成绩日志	<code>services/score_service.py</code> 、 <code>pages/score_manage.py</code>

功能	当前实现	主要文件
基础资料维护	管理学生、教师资料并同步创建账号	services/student_service.py、services/teacher_service.py

2.3 数据需求

系统需要保存的数据分为五类。第一类是组织结构数据，包括 department、major、classroom。第二类是用户身份数据，包括 user_account、user_session、student、teacher、admin_profile。第三类是教学安排数据，包括 semester、course、course_offering、course_schedule 和 course_prerequisite。第四类是选课成绩数据，即 enrollment。第五类是审计数据，即 score_change_log。

2.4 业务规则

当前已经实现的业务规则包括：

1. 重复选课限制：select_course_tx() 会锁定并检查同一学生、同一教学班的已有选课记录，enrollment 表上的 UNIQUE(student_id, offering_id) 作为数据库层兜底。
2. 容量限制：trg_enrollment_guard 在锁定 course_offering 行后统计 status = 'selected' 的人数，并与 max_capacity 比较。
3. 行级锁保护：选课函数和选课触发器均使用 SELECT...FORUPDATE 锁定目标 course_offering 行。
4. 时间冲突检查：trg_enrollment_guard 通过 course_schedule 的 weekday、start_section、end_section 判断同学期区间重叠。
5. 先修课限制：trg_enrollment_guard 通过 course_prerequisite 和历史 completed 成绩判断先修课是否达到 60 分。
6. 退课规则：只能退本人 selected 状态且未录入成绩的记录；数据库触发器禁止已完成或已有成绩的记录退课。
7. 成绩权限：教师只能维护自己负责的班次，管理员可以维护所有班次。
8. 成绩范围：数据库 CHECK 与服务层均限制成绩为 0 到 100。
9. 成绩审计：trg_score_change_log 在 final_score 变化后自动写入 score_change_log，操作者由成绩更新事务传入。
10. 角色档案一致性：学生、教师、管理员档案由触发器检查其 user_id 对应账户角色是否匹配。
11. 教学班与教室容量一致性：trg_offering_capacity_check 和 trg_classroom_capacity_update_check 维护 max_capacity 与教室容量之间的跨表约束。

仍需进一步完善的规则包括：同一学生同一学期选择同一课程不同教学班的限制、数据库级角色授权、备份恢复流程以及多连接并发压测。

2.5 数据字典草案

表 2-2 核心数据字典草案

数据项	来源表	含义
student_id	student	学生业务编号，学生主键。
teacher_id	teacher	教师业务编号，教师主键。
course_id	course	课程编号，表示课程定义。
offering_id	course_offering	教学班代理主键，表示某学期某教师开设的具体课次。
schedule_id	course_schedule	上课时间片主键。
enrollment_id	enrollment	选课记录主键。
final_score	enrollment	学生在某教学班的最终成绩。
token_hash	user_session	持久登录 token 的哈希值。

第 3 章 概念结构设计

3.1 实体抽取

根据当前 `init.sql` 反向抽取，系统实体可以分为组织结构、用户身份、教学安排、选课成绩和审计记录五组。

- 组织结构实体：院系 `department`、专业 `major`、教室 `classroom`。
- 用户身份实体：统一账号 `user_account`、学生 `student`、教师 `teacher`、管理员资料 `admin_profile`、用户会话 `user_session`。
- 教学安排实体：学期 `semester`、课程 `course`、开课班次 `course_offering`、上课时间片 `course_schedule`、先修关系 `course_prerequisite`。
- 选课成绩实体：选课记录 `enrollment`。
- 审计实体：成绩修改日志 `score_change_log`。

3.2 属性设计

课程和开课班次是本项目概念设计的关键。课程 `course` 保存课程号、课程名、课程类型、学分、学时、所属院系和状态；开课班次 `course_offering` 保存课程、学期、教师、教室、容量上限和班次状态。二者分离后，同一课程可在不同学期、不同教师、不同教室多次开设，而不会重复保存课程名称、学分、学时等基础信息。

本次 3NF 重构后，旧的自由文本上课时间不再保存在 `course_offering` 中，而是由 `course_schedule(schedule_id, offering_id, weekday, start_section, end_section)` 表示。一个教学班可对应多个时间片，便于后续进行时间冲突检查。

3.3 联系与基数

主要联系如下：

- 院系与专业、教师、课程均为 1:N。
- 统一账号与学生、教师、管理员资料为 1:0..1。
- 课程与开课班次为 1:N。
- 学期与开课班次为 1:N。
- 教师与开课班次为 1:N。
- 教室与开课班次为 1:N。
- 开课班次与上课时间片为 1:N。
- 学生与开课班次是 M:N，通过 `enrollment` 转换为两个 1:N 联系。
- 课程与课程之间存在先修自关联，通过 `course_prerequisite` 转换。

3.4 ER 图

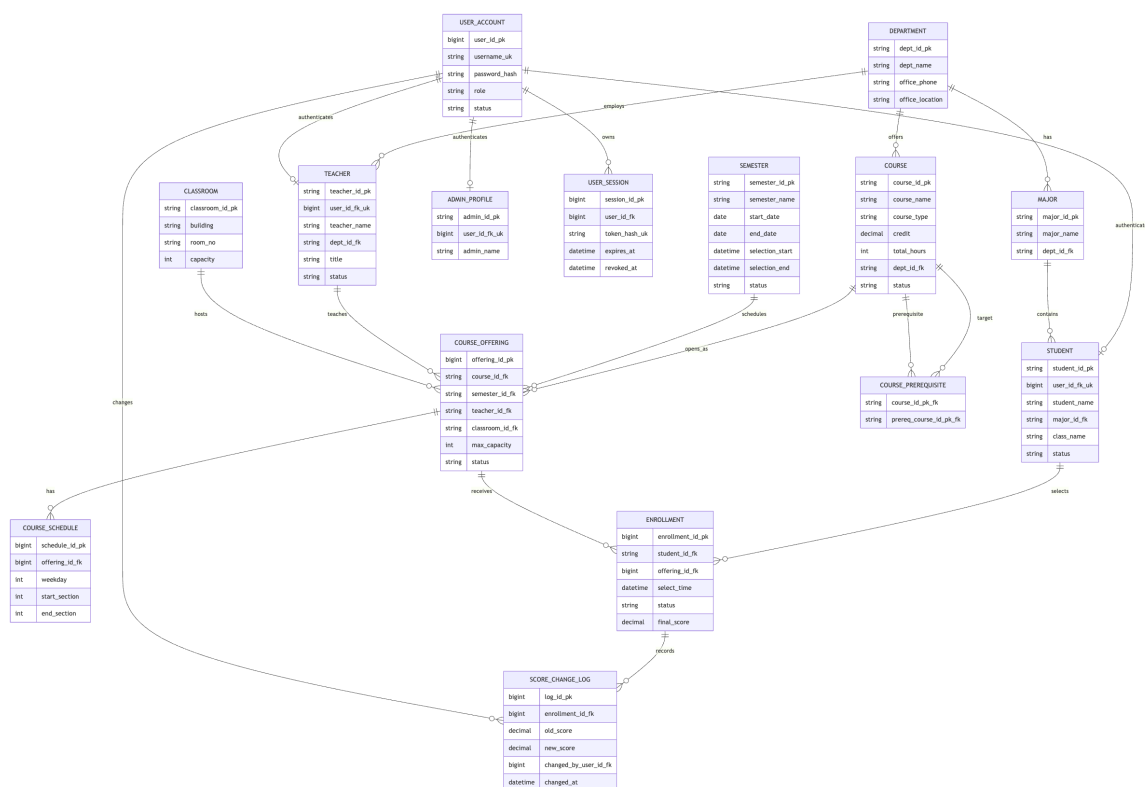


图 3-1 选课系统概念结构 ER 图

图 3-1 展示了系统核心实体和联系。图中字段名后缀 **_pk**、**_fk**、**_uk** 分别表示主键、外键和唯一约束。由于 Mermaid ER 图语法限制，复合约束以字段名后缀方式标注。

3.5 从业务语义到概念模型

学生并不是直接选择“课程定义”，而是选择某个学期中由某位教师开设的具体教学班。因此，将学生与课程直接建模为 M:N 会丢失教师、学期、教室、容量和时间等重要语义。本项目使用 `course_offering` 表示教学班，再用 `enrollment` 表示学生选择教学班这一联系，既符合现实业务，也便于记录选课状态和成绩。

先修关系是课程与课程之间的自关联。如果直接在 `course` 中保存一个先修课程字段，只能表达一个先修课程且不利于扩展。当前使用 `course_prerequisite(course_id, prereq_course_id)`，可以表达一门课程有多个先修课，也可以表达一门课被多门后续课依赖。

第 4 章 逻辑结构设计

4.1 关系模式

由概念模型转换得到的主要关系模式如下。表名和字段均依据 `opengauss_setup/sql/init.sql`。

- `Department(dept_id,dept_name,office_phone,office_location)`
- `Major(major_id,major_name,dept_id)`
- `UserAccount(user_id,username,password_hash,role,status,last_login_at,created_at)`
- `UserSession(session_id,user_id,token_hash,created_at,expires_at,revoked_at,last_seen_at)`
- `Student(student_id,user_id,student_name,gender,birth_date,enroll_year,major_id,class_name,phone,email,status)`
- `Teacher(teacher_id,user_id,teacher_name,gender,dept_id,title,phone,email,status)`
- `AdminProfile(admin_id,user_id,admin_name,phone)`
- `Semester(semester_id,semester_name,start_date,end_date,selection_start,selection_end,status)`
- `Course(course_id,course_name,course_type,credit,total_hours,dept_id,description,status)`
- `Classroom(classroom_id,building,room_no,capacity)`
- `CourseOffering(offering_id,course_id,semester_id,teacher_id,classroom_id,max_capacity,status)`
- `CourseSchedule(schedule_id,offering_id,weekday,start_section,end_section)`
- `CoursePrerequisite(course_id,prereq_course_id)`
- `Enrollment(enrollment_id,student_id,offering_id,select_time,status,final_score,remark)`
- `ScoreChangeLog(log_id,enrollment_id,old_score,new_score,changed_by_user_id,changed_at,reason)`

4.2 关系模式、主键、外键与唯一约束

`init.sql` 为所有核心表设置主键，并通过外键维护主要参照完整性。表 4-1 汇

总结了当前主要关系模式的键约束。

表 4-1 关系模式、主键、外键与唯一约束

关系模式	主键	主要外键	唯一约束 / 候选键
department	dept_id	无	UNIQUE(dept_name)
major	major_id	dept_id->department.dept_id	UNIQUE(dept_id, major_name)
user_account	user_id	无	UNIQUE(username)
user_session	session_id	user_id->user_account.user_id	UNIQUE(token_hash)
student	student_id	user_id->user_account.user_id; major_id->major.major_id	UNIQUE(user_id)
teacher	teacher_id	user_id->user_account.user_id; dept_id->department.dept_id	UNIQUE(user_id)
admin_profile	admin_id	user_id->user_account.user_id	UNIQUE(user_id)
semester	semester_id	无	无显式业务唯一约束
course	course_id	dept_id->department.dept_id	无显式业务唯一约束
classroom	classroom_id	无	UNIQUE(building, room_no)
course_offering	offering_id	course_id->course.course_id; semester_id->semester.semester_id; teacher_id->teacher.teacher_id; classroom_id->classroom.classroom_id	无显式复合业务唯一约束
course_schedule	schedule_id	offering_id->course_offering.offering_id	UNIQUE(offering_id, weekday, start_section, end_section)
course_prerequisite	course_id, prerequisite_course_id	两列均引用 course.course_id	复合主键即业务候选键
enrollment	enrollment_id	student_id->student.student_id; offering_id->course_offering.offering_id	UNIQUE(student_id, offering_id)
score_change_log	log_id	enrollment_id->enrollment.enrollment_id; changed_by_user_id->user_account.user_id	无显式业务唯一约束

4.3 多对多关系转换

学生与开课班次之间是 M:N。若直接把课程列表保存在学生表中，会产生列表型字段并破坏 1NF；若把学生列表保存在开课班次表中，同样不利于查询和约束。当前设计使用 enrollment 作为中间表，把 M:N 转换为：

- student 与 enrollment 的 1:N；
- course_offering 与 enrollment 的 1:N。

同时, enrollment 自身具有 select_time、status、final_score 和 remark 等联系属性, 因此它不只是简单连接表, 也是描述选课过程和成绩结果的业务实体。UNIQUE(student_id, offering_id) 防止同一学生重复选择同一教学班。

4.4 课程定义与开课班次分离

当前系统已经正确分离 course 与 course_offering。course 保存稳定的课程定义, 例如课程名称、课程类型、学分、总学时和所属学院; course_offering 保存某课程在某学期由某教师、某教室开设的具体教学班次。若二者不分离, 则同一课程每次开班都要重复保存课程名、学分、学时等信息, 课程定义调整时容易产生更新异常。

semester、teacher 和 classroom 也独立成表, 使 course_offering 只保存外键和班次自身属性 max_capacity、status。该设计有利于后续支持同一课程多学期、多教师、多教学班开设。

4.5 多值属性与自关联关系处理

course_schedule 是 course_offering 多值时间属性的拆分结果。一个教学班可以有多个上课时间片, 因此逻辑设计中将“上课时间”从 course_offering 中拆出为独立关系, 每行表示一个 weekday, start_section, end_section 组合。这种设计避免了在开课表中保存 schedule_text 这类复合文本字段。

course_prerequisite 是 course 对 course 的自关联关系。它使用 course_id 和 prereq_course_id 作为复合主键, 分别引用同一张 course 表, 用于表达一门课程可以有多门先修课程、一门课程也可以作为多门课程的先修课程。

4.6 逻辑关系模型图

4.7 参照完整性

当前系统使用外键保证主要参照完整性。例如, 选课记录必须引用存在的学生和教学班; 开课班次必须引用存在的课程、学期和教师; 上课时间片必须引用存在的开课班次; 成绩变更日志必须引用存在的选课记录和操作账号。值得注意的是, user_session.user_id 和 course_schedule.offering_id 使用 ONDELETECASCADE, 分别用于在删除账号时清理会话、在删除无选课记录的教学班时清理时间片。

部分跨表业务规则无法仅靠普通外键表达, 当前已经通过触发器补充其中的核心规则: course_offering.max_capacity<=classroom.capacity 由教学班和教室容量触发器维护, 账户角色与具体档案表之间的一致性由角色档案触发器维护。仍需改进的是同一学生同一学期不重复选择同一课程的不同教学班, 这类规则需要在后续通过跨表触发器、培养方案规则表或更细的业务键设计实现。

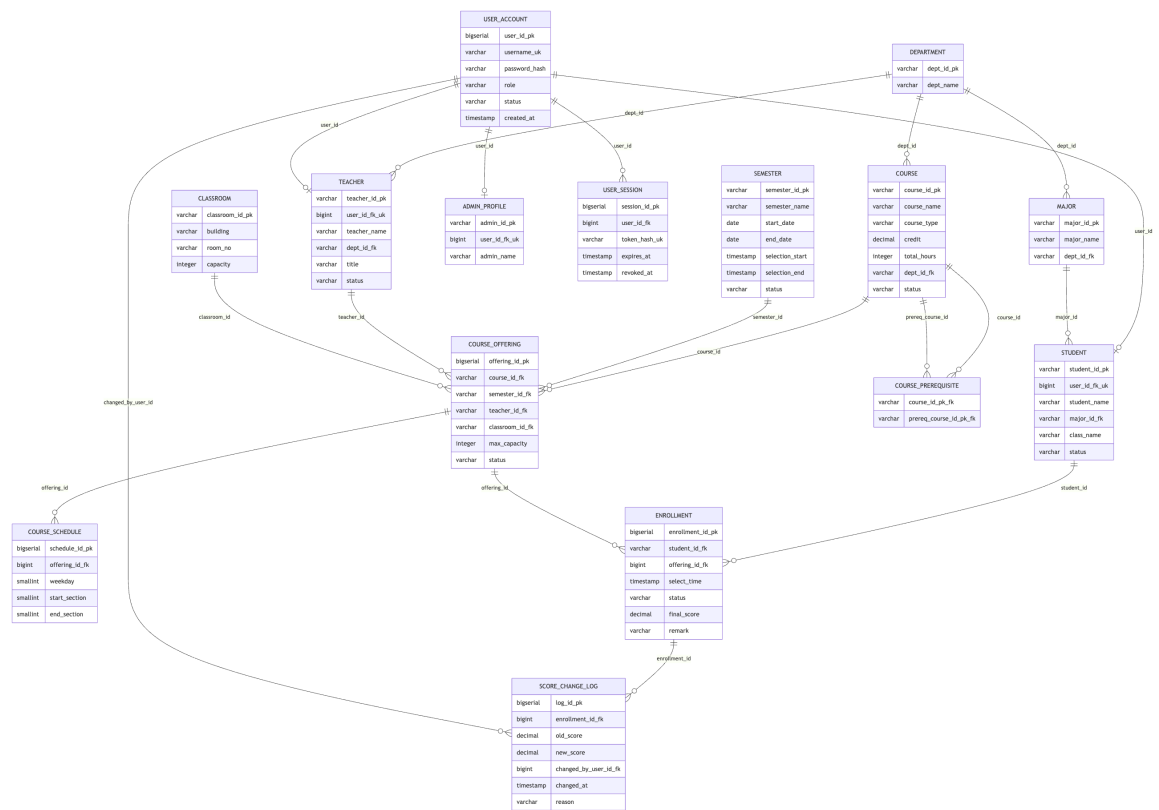


图 4-1 选课系统逻辑关系模型图

第 5 章 规范化设计与范式分析

5.1 规范化分析对象与方法

规范化设计的目标是在保证业务语义完整表达的前提下，减少数据冗余，避免插入异常、删除异常和更新异常。本章不单独停留在范式概念说明，而是以系统当前数据库 schema 为对象，对核心关系模式进行函数依赖、候选键、第一范式、第二范式、第三范式和 BCNF 分析。主要依据为 opengauss_setup/sql/init.sql、opengauss_setup/sql/migrate_3nf_20260608.sql 和 opengauss_setup/sql/add_user_session.sql。

当前系统已经将组织结构、账户认证、角色档案、课程定义、开课班次、上课时间、选课记录和成绩变更日志拆分为相对独立的关系模式。该设计体现了从 ER 模型到关系模型的转换过程，也为完整性约束、事务控制和后续维护提供了基础。需要特别说明的是，本章只把 SQL 中已经存在的表、字段、主键、外键、唯一约束、CHECK 约束、触发器和视图作为“当前实现”；绩点规则表、行政班实体等尚未落地的机制仍作为后续改进讨论。

5.2 当前关系模式与主键、候选键、外键

从业务模块看，当前数据库可以分为四类关系模式。

- (1) 组织结构模块：department、major；
- (2) 账户与角色模块：user_account、user_session、student、teacher、admin_profile；
- (3) 课程教学模块：semester、course、classroom、course_offering、course_schedule、course_prerequisite；
- (4) 选课成绩模块：enrollment、score_change_log。

这种划分避免了把登录信息、学生档案、课程定义、具体开课、选课关系和成绩日志混合在单张大表中，从整体上降低了数据冗余和更新异常风险。表 5-1 按关系模式列出主键、候选键、主要函数依赖和范式判断。

表 5-1 核心关系模式的函数依赖与范式审计

关系模式	主键 / 候选键	主要函数依赖	范式判断	说明
department	dept_id	dept_id → dept_name, office_phone, office_location	3NF, 具有 BCNF 倾向	已补充 uq_department_name, 学院名称也成为显式业务唯一决定因素。

关系模式	主键 / 候选键	主要函数依赖	范式判断	说明
major	major_id	major_id → major_name, dept_id	3NF, 具有 BCNF 倾向	通过 dept_id 引用学院, 且已补充 uq_major_dept_name 避免同院系专业重名。
user_account	user_id; username	user_id → username, password_hash, role, status; username → user_id	3NF/BCNF 倾向明显	username 有唯一约束, 是登录业务候选键。
user_session	session_id; token_hash	session_id → user_id, token_hash, created_at, expires_at, revoke_at; token_hash → session_id	3NF/BCNF 倾向明显	会话独立成表, 避免在账户表中保存多个 token, 符合 1NF。
student	student_id; user_id	student_id → user_id, student_name, gender, major_id, class_name, status; user_id → student_id	3NF, BCNF 需结合业务语义	学生档案与账户分离; major_id 外键避免保存专业和院系详细信息。
teacher	teacher_id; user_id	teacher_id → user_id, teacher_name, dept_id, title, status; user_id → teacher_id	3NF, BCNF 需结合业务语义	教师档案独立, 避免账户表膨胀, 并通过 dept_id 引用学院。
admin_profile	admin_id; user_id	admin_id → user_id, admin_name, phone; user_id → admin_id	3NF/BCNF 倾向明显	管理员资料与认证信息分离, user_id 唯一表达一对一关系。
semester	semester_id	semester_id → semester_name, start_date, end_date, selection_start, selection_end, status	3NF, 具有 BCNF 倾向	学期日期和选课窗口独立保存, 避免在开课表中重复。
course	course_id	course_id → course_name, course_type, credit, total_hours, dept_id, status	3NF, BCNF 需结合业务语义	保存课程定义, 不保存具体学期、教师或教室。
classroom	classroom_id	classroom_id → building, room_no, capacity	3NF, BCNF 需补充候选键确认	教室容量独立保存; 已补充 uq_classroom_location 避免重复教室实体。
course_offering	offering_id	offering_id → course_id, semester_id, teacher_id, classroom_id, max_capacity, status	3NF, 具有 BCNF 倾向	表示某课程在某学期由某教师开设的教学班, 不再保存 schedule_text 和 selected_count。
course_schedule	schedule_id; offering_id; weekday; start_section; end_section	schedule_id → offering_id, weekday, start_section, end_section	1NF/2NF/3NF 良好	将上课时间拆为结构化时间片, 业务唯一约束防止同一班次重复时间片。
course_prerequisite	course_id, prerequisite_id	无非主属性	2NF/3NF/BCNF 倾向明显	复合主键表, 无非主属性, 因此无部分依赖和传递依赖。
enrollment	enrollment_id; student_id; offering_id	enrollment_id → student_id, offering_id, select_time, status, final_score, remark; student_id, offering_id → enrollment_id	3NF, 具有 BCNF 倾向	选课关系表, 同时保存状态、时间和成绩等联系属性; 不再保存 gpa_point。
score_change_log	log_id	log_id → enrollment_id, old_score, new_score, changed_by_user_id, changed_at, reason	3NF, 具有 BCNF 倾向	成绩修改历史独立保存, 并由成绩更新触发器自动写入审计日志。

5.3 核心函数依赖分析

根据当前 SQL 结构, 主要函数依赖可以分为实体标识依赖、业务候选键依赖和联系属性依赖三类。

第一类是实体标识依赖。例如，`dept_id` 决定学院名称、办公电话和办公地点；`course_id` 决定课程名称、课程类型、学分、学时和所属学院；`classroom_id` 决定楼栋、房间号和容量。这类依赖由主键直接表达。

第二类是业务候选键依赖。例如，`user_account.username` 有唯一约束，因此 `username` 可以决定 `user_id` 及账户状态；`user_session.token_hash` 有唯一约束，因此 `token` 摘要可以决定对应会话；`student.user_id`、`teacher.user_id`、`admin_profile.user_id` 均具有唯一约束，用于表达账户与角色档案之间的一对一关系。

第三类是联系属性依赖。典型代表是 `enrollment: student_id, offering_id` 共同决定一条选课记录，`select_time`、`status`、`final_score`、`remark` 都依赖于“一名学生选择一个教学班”这一完整联系，而不是只依赖学生或只依赖教学班。

本次 3NF 迁移重点删除了若干派生依赖风险：`course_offering.selected_count` 可由 `enrollment` 聚合得到；`enrollment.gpa_point` 可由 `final_score` 和绩点规则推导；`course_offering.schedule_text` 则把多个时间片压入一个文本字段。这些字段不再保存在基础表中。

5.4 第一范式分析：属性原子性与结构化时间片

第一范式要求关系中的每个属性都是不可再分的原子值，不能在一个字段中保存列表、集合或重复组。当前系统最典型的 1NF 改造体现在上课时间设计中。

旧设计如果将上课时间保存为 `course_offering.schedule_text`，例如“周一 1-2 节 / 周三 3-4 节”，则一个字段中同时包含多个时间片。该设计虽然便于页面展示，但不利于数据库按星期和节次进行查询，也不利于选课时间冲突检查。当前设计将其拆分为 `course_schedule` 表，每条记录只表示一个教学班的一个上课时间片，包括 `weekday`、`start_section` 和 `end_section`。这样，每个字段都具有清晰的原子语义，满足第一范式要求。

`migrate_3nf_20260608.sql` 也体现了这一改造路径：脚本先创建 `course_schedule`，再使用 `REGEXP_SPLIT_TO_TABLE` 将旧的 `schedule_text` 按分隔符解析为多行结构化时间片，最后删除 `course_offering.schedule_text`。迁移脚本同时提醒，如果历史数据中存在自定义格式的上课时间文本，需要在删除旧字段前人工检查解析结果。

此外，`user_session` 表没有在 `user_account` 中保存多个登录 `token`，而是将每次会话独立成行；`course_prerequisite` 没有在 `course` 中使用字符串列表保存先修课程，而是使用自关联中间表表达课程之间的先修关系。这些设计都避免了多值字段，提高了结构化查询能力。

5.5 第二范式分析：复合键与部分函数依赖

第二范式要求在满足第一范式的基础上，非主属性必须完全函数依赖于候选键，而不能只依赖复合键的一部分。对于使用单属性代理主键的关系模式，部分函数依赖问题通常不明显；本系统中更需要重点分析的是具有复合业务键的关系。

`course_prerequisite` 的主键为 `course_id, prereq_course_id`，表中没有额外非主属性，因此不存在非主属性只依赖 `course_id` 或只依赖 `prereq_course_id` 的问题。该表的意义是把课程对课程的先修多对多关系转换为关系模式。

`course_schedule` 虽然使用 `schedule_id` 作为代理主键，但其业务唯一约束为 `offering_id, weekday, start_section, end_section`。该组合共同决定一个具体时间片，表中没有诸如课程名称、教师姓名、教室容量等只依赖 `offering_id` 的冗余字段，因此不存在明显部分依赖。

`enrollment` 使用 `enrollment_id` 作为代理主键，同时使用 `student_id, offering_id` 作为业务候选键。选课时间、选课状态、最终成绩和备注都描述一次具体选课关系，而不是只描述学生或只描述教学班。因此这些属性依赖完整的选课关系，满足第二范式要求。

5.6 第三范式分析：传递依赖、派生字段与表分解

第三范式要求在满足第二范式的基础上，非主属性之间不应存在传递函数依赖。当前系统通过多处表分解减少了传递依赖。

在组织结构设计中，`student` 表只保存 `major_id`，不直接保存学院名称、学院电话或学院办公地点；`major` 表保存 `dept_id`；`department` 表保存学院详细信息。这样避免了在学生表中出现 `student_id → major_id → dept_name` 的传递依赖。类似地，`teacher` 和 `course` 也只通过 `dept_id` 引用学院，而不重复保存学院详细信息。

在课程设计中，`course` 保存课程定义，`course_offering` 保存具体开课班次，`semester`、`teacher`、`classroom` 分别保存学期、教师和教室信息。开课班次表只保存这些实体的外键，而不重复保存课程名称、教师姓名、学期起止日期或教室容量，从而减少了更新异常。

账户设计也体现了表分解思想。`user_account` 保存登录账号、密码哈希、角色、状态和最近登录时间；`student`、`teacher`、`admin_profile` 分别保存不同角色的业务档案，并通过唯一的 `user_id` 与账户建立一对一关系。这种设计避免了把学生、教师和管理员的差异化字段堆在同一张账户表中。本轮已经通过 `trg_student_role_check`、`trg_teacher_role_check` 和 `trg_admin_role_check` 强制档案表引用的账户角色与档案类型一致。

第三范式改造中最重要的部分是删除派生字段。旧设计中的 `course_offering.selected_count` 可以通过 `enrollment` 表按教学班和选课状态统计得到。如果同时

在开课表中保存已选人数，则每次选课、退课或状态变更都必须同步更新该字段，一旦事务失败或并发控制不当，就可能出现容量统计不一致。当前 3NF 迁移删除该字段，体现了消除派生冗余的设计思路。服务层需要显示已选人数时，通过 `COUNT(*)` 或子查询动态计算；如果未来数据量很大，可以考虑视图、物化视图或受控冗余，而不是简单恢复 `selected_count`。

类似地，旧设计中的 `enrollment.gpa_point` 可以由 `final_score` 和绩点换算规则推导。如果直接存储绩点，当成绩被修改时，绩点也必须同步修改，否则会产生更新异常。因此迁移脚本删除 `gpa_point`。当前 `services/score_service.py` 和 `services/course_service.py` 在查询时用 CASE 表达式生成展示用绩点。如果后续系统确实需要管理不同年份、不同课程或不同学院的绩点换算规则，应设计 `grade_policy` 或 `grade_scale` 表保存规则，再通过查询或视图计算绩点。

`score_change_log` 的存在不属于上述派生字段冗余。它保存的是成绩修改事件历史，包含旧成绩、新成绩、修改人、修改时间和原因。若只在 `enrollment` 中覆盖 `final_score`，历史成绩会丢失；独立日志表反而增强了审计能力。本轮已经实现 `trg_score_change_log`，成绩变更由数据库触发器自动写入日志。

5.7 BCNF 分析：决定因素与候选键

BCNF 要求每一个非平凡函数依赖的决定因素都必须是候选键。与 3NF 相比，BCNF 对决定因素要求更严格，因此不能在证据不足时简单宣称所有关系模式都完全满足 BCNF。

在当前 SQL 已声明的主键、唯一约束和主要业务语义下，多数核心关系模式的决定因素是主键或候选键，因此具有较好的 BCNF 倾向。比如，`user_account` 中 `user_id` 和 `username` 都可以作为候选键；`student`、`teacher`、`admin_profile` 中 `user_id` 也通过唯一约束表达与账户的一对一关系；`enrollment` 中 `student_id`、`offering_id` 作为业务候选键防止重复选课；`course_schedule` 中 `offering_id`、`weekday`、`start_section`、`end_section` 表示唯一时间片。

但 BCNF 的严格判断依赖业务语义。本轮已经将 `dept_name`、`building`、`room_no`、`dept_id`、`major_name` 这三类业务唯一性显式声明为 `UNIQUE` 约束，使这些业务决定因素成为候选键。由此，`department`、`major` 和 `classroom` 在当前业务语义下的 BCNF 说明更加充分。

因此，本系统当前设计在主键和已声明唯一约束范围内具有较好的 BCNF 倾向。但如果未来继续增加新的业务语义，例如正式行政班编码、绩点规则版本、跨学期培养方案等，仍需要同步声明新的候选键或拆分新的关系模式。

5.8 从旧设计到 3NF 的迁移说明

`migrate_3nf_20260608.sql` 是本报告中体现规范化改造的重要证据。该脚本先

删除旧的选课触发器和函数，再创建 `course_schedule`，迁移历史上课时间文本，最后删除反范式风险字段，并创建补偿查询性能的索引。表 5-2 对旧字段和当前改造进行对比。

表 5-2 旧设计字段与 3NF 改造说明

旧设计	存在问题	当前改造	对应范式意义
<code>course_offering.schedule_text</code>	一个文本字段包含多个上课时间片，例如“周一 1-2 节 / 周三 3-4 节”，不便于时间冲突检查，也不符合属性原子化要求。	迁移到 <code>course_schedule(offering_id, weekday, start_section, end_section)</code> ，迁移脚本解析旧文本后删除该字段。	增强 1NF，支持结构化查询。
<code>course_offering.selected_count</code>	已选人数由 <code>enrollment</code> 统计得到，是派生字段，容易在插入、退课和并发选课时产生更新异常。	删除 <code>selected_count</code> ，通过 <code>enrollment</code> 统计或视图获得已选人数。	减少冗余，增强 3NF，一致性更好。
<code>enrollment.gpa_point</code>	绩点可由 <code>final_score</code> 和绩点规则推导，直接保存可能与最终成绩不一致。	删除 <code>gpa_point</code> ，后续可通过 <code>grade_policy</code> 、 <code>grade_scale</code> 或视图计算。	避免派生数据冗余，减少更新异常。
<code>trg_enrollment_insert / trg_enrollment_update</code>	旧设计可能依赖触发器维护 <code>selected_count</code> ，一旦触发器失败或并发复杂，容易产生一致性问题。	迁移脚本删除相关触发器和函数，当前转为消除冗余字段。	从“维护冗余字段”转向“消除冗余字段”。

5.9 当前设计中仍可改进的规范化问题

虽然当前设计已经较好地体现了规范化思想，但仍存在若干可以进一步完善的地方。

首先，`class_name` 当前作为文本字段保存在 `student` 表中。如果班级只是展示标签，该设计可以接受；但如果班级还具有辅导员、年级、所属专业、班级人数等独立属性，则应新增 `class` 表，并在 `student` 中保存 `class_id`，避免班级信息重复。

其次，本轮新增的 `uq_department_name`、`uq_major_dept_name` 和 `uq_classroom_location` 已经把学院名称、同院系专业名称、楼栋房间号等业务唯一性固化到数据库层。这些约束不仅防止重复数据，也能让业务决定因素显式成为候选键，有助于更严格地讨论 BCNF。

再次，`course_prerequisite` 当前可以表达课程先修关系。本轮已经增加 `chk_prereq_not_self`，避免课程把自己设置为自己的先修课；同时通过 `trg_prerequisite_no_cycle` 使用递归查询检测先修课程环路。

最后，如果后续需要绩点计算，应新增 `grade_policy` 或 `grade_scale` 表，而不是恢复 `enrollment.gpa_point`；如果需要高性能课程容量统计，可以用视图、物化视图或受控冗余，而不是直接恢复 `selected_count`。这些方案都需要在性能和一致

性之间进行权衡。

5.10 本章小结

本章从函数依赖和候选键出发，对系统核心关系模式进行了规范化审计。当前设计通过组织结构分解、账户与角色档案分离、课程定义与开课班次分离、上课时间片结构化、选课关系独立建模和成绩日志独立保存，较好地避免了多值字段、部分依赖和传递依赖。尤其是 `course_schedule` 的引入以及 `schedule_text`、`selected_count`、`gpa_point` 的删除，体现了从实现便利型设计向规范化设计的演进过程。本轮新增的业务唯一约束、触发器和视图没有破坏 3NF：视图只是查询层封装，触发器用于维护跨表完整性。后续系统可以继续通过班级实体、绩点规则表和培养方案规则表来提高数据库设计的严谨性。

第 6 章 完整性约束、触发器与检查机制

6.1 完整性约束设计目标

本章依据 `opengauss_setup/sql/init.sql`、`migrate_triggers_constraints_20260608.sql` 和 `test_triggers_constraints.sql`，说明当前系统已经落地的数据库完整性机制。完整性设计目标包括：实体可唯一标识、外键引用合法、业务取值范围合法、角色档案与账户角色一致、先修课程关系无自指和环路、选课过程不超容量且不发生时间冲突、成绩修改可以审计追踪。

与旧版本通过触发器维护 `selected_count` 的思路不同，当前系统在 3NF 迁移后删除了 `selected_count` 派生字段。本系统不再通过触发器同步该字段，而是通过 `enrollment` 表和视图统计当前已选人数。触发器的职责转向跨表业务完整性检查和审计记录维护，例如选课容量检查、时间冲突检查、先修课程合法性检查以及成绩变更日志写入。

6.2 实体完整性：主键约束

实体完整性主要通过主键实现。当前 `init.sql` 为所有核心表设置主键，例如 `department.dept_id`、`student.student_id`、`teacher.teacher_id`、`course.course_id`、`course_offering.offering_id`、`course_schedule.schedule_id`、`enrollment.enrollment_id`。这些主键保证每个实体或业务事件可以被唯一标识。

6.3 参照完整性：外键约束

参照完整性通过外键约束实现。典型外键包括：`major.dept_id` 引用 `department.dept_id`，`student.user_id` 引用 `user_account.user_id`，`course_offering.course_id` 引用 `course.course_id`，`course_schedule.offering_id` 引用 `course_offering.offering_id`，`enrollment.student_id` 引用 `student.student_id`，`score_change_log.enrollment_id` 引用 `enrollment.enrollment_id`。其中 `user_session.user_id` 和 `course_schedule.offering_id` 使用 `ONDELETECASCADE`，用于自动清理会话和教学班时间片。

6.4 用户自定义完整性：UNIQUE、CHECK、NOT NULL、DEFAULT

本轮新增的 `migrate_triggers_constraints_20260608.sql` 在原有主键、外键、`NOT NULL`、`DEFAULT` 和 `CHECK` 约束基础上，进一步补充了业务唯一约束和合法性检查。表 6-1 总结了主要约束。

表 6-1 主要完整性约束

表	约束	业务含义
department	uq_department_name	学院名称唯一，避免同一学院重复建档。
major	uq_major_dept_name	同一学院内专业名称唯一。
classroom	uq_classroom_location	同一楼栋同一房间号只能对应一个教室实体。
course_prerequisite	chk_prereq_not_self	课程不能把自己作为自己的先修课程。
semester	chk_semester_date_range	学期起止日期、选课窗口起止时间必须合法。
user_session	chk_user_session_time_range	会话过期时间必须晚于创建时间，撤销时间不能早于创建时间。
course_schedule	chk_schedule_section_range	节次范围限定为 1 到 12，并要求结束节次不早于开始节次。
enrollment	UNIQUE(student_id, offering_id)	防止同一学生重复选择同一教学班。

教学班容量与教室容量的关系属于跨表约束，普通 CHECK 无法查询另一张表，因此本系统使用触发器实现。迁移脚本执行前先审计并修正了已有样例数据中 MA101 班次容量大于 M201 教室容量的问题。

6.5 跨表业务完整性与触发器设计

本轮已经实现数据库触发器，触发器定义集中在 opengauss_setup/sql/migrate_triggers_constraints_20260608.sql。这些触发器不是用于维护 selected_count，而是用于角色档案一致性、先修关系合法性、选课完整性、教室容量一致性和成绩审计。表 6-2 给出触发器清单。

表 6-2 数据库触发器实现清单

触发器名称	作用表	触发时机	检查或维护内容	对应课程知识点	实现状态
trg_student_role_check	student	插入或修改前	user_id 学生档案必须引用 role='student' 的账户	跨表用户自定义完整性	已实现

触发器名称	作用表	触发时机	检查或维护内容	对应课程知识点	实现状态
trg_teacher_role_check	teacher	插入或修改 user_id 前	教师档案必须引用 role='teacher' 的账户	子类型一致性	已实现
trg_admin_role_check	admin_profile	插入或修改 user_id 前	管理员档案必须引用 role='admin' 的账户	角色完整性	已实现
trg_prerequisite_no_cycle	course_prerequisite	插入或更新前	禁止自指先修课, 并用递归查询检测环路	递归查询与复杂完整性	已实现
trg_offering_capacity_check	course_offering	插入或改容量、教室前	教学班容量不能超过教室容量, 且不能低于当前已选人数	跨表约束	已实现
trg_classroom_capacity_update_check	classroom	更新 capacity 前	降低教室容量时不能小于已有教学班容量	更新约束	已实现
trg_enrollment_guard	enrollment	插入或改学生、班次、状态前	检查学生状态、班次状态、选课窗口、容量、时间冲突、先修课和退课限制	主动规则、事务与锁	已实现
trg_score_change_log	enrollment	更新 final_score 后	自动写入 score_change_log, 记录旧成绩、新成绩、操作者和原因	审计触发器	已实现

6.6 选课完整性触发器

trg_enrollment_guard 是选课系统最核心的数据库层完整性保护。对 status='selected' 的插入或恢复选课, 它会锁定目标 course_offering 行, 然后检查当前已选人数是否小于 max_capacity。已选人数通过 enrollment 聚合统计得到, 系统没有恢复 course_offering.selected_count。

该触发器还基于 course_schedule 检查同一学期内的时间冲突, 区间重叠条件为:

```

1 target.weekday = existing.weekday
2 AND existing.start_section <= target.end_section
3 AND existing.end_section >= target.start_section

```

先修课程检查基于 course_prerequisite 和历史 enrollment 记录。系统采用 60 分作为通过先修课程的业务规则, 即学生必须存在 status='completed' 且 final_score>=60 的先修课程记录。退课时, 触发器禁止已完成课程或已有成绩记录改为 dropped。

6.7 成绩变更审计触发器

trg_score_change_log 在 enrollment.final_score 更新后触发。当旧成绩和新成绩不同, 触发器自动向 score_change_log 插入日志。由于日志表要求 changed_by_user_id 非空, 业务层必须在同一事务内设置当前操作者:

```

1 SELECT set_config('app.current_user_id', '<user_id>', true);
2 SELECT set_config('app.score_change_reason', '<reason>', true);
3 UPDATE enrollment
4 SET final_score = 88, status = 'completed'

```

```
5 WHERE enrollment_id = 1;
```

如果业务层没有设置 `app.current_user_id`，触发器会拒绝成绩更新。这比单纯在应用层手工插入日志更可靠，因为任何直接更新 `final_score` 的数据库操作都会被同一个审计规则约束。

6.8 角色档案一致性触发器

`student`、`teacher` 和 `admin_profile` 虽然都通过 `user_id` 外键引用 `user_account`，但普通外键只能保证账户存在，不能保证角色匹配。因此本系统实现了通用函数 `check_user_role_fn(p_user_id,p_expected_role)`，并由三个档案触发器复用。测试脚本已经验证，将教师账户插入学生档案、将学生账户插入教师档案都会被拒绝。

6.9 先修课程合法性触发器

`chk_prereq_not_self` 可以阻止 `course_id=prereq_course_id` 的直接自指，但不能发现更复杂的 A 依赖 B、B 依赖 C、C 又依赖 A 的环路。本系统通过 `trg_prerequisite_no_cycle` 在插入或更新先修关系前执行递归查询，沿着先修链查找是否会回到当前课程。如果形成环路，触发器抛出异常并阻止写入。

6.10 应用层校验与数据库触发器的关系

当前应用层仍保留友好的错误提示和页面交互检查，例如 `services/selection_service.py` 会调用数据库函数 `select_course_tx()`，`services/score_service.py` 会在事务内设置审计上下文。数据库触发器承担兜底一致性责任：即使绕过前端或服务层，只要直接写数据库，也必须通过容量、时间冲突、先修课程、角色档案和成绩审计规则。

这种分层体现了数据库完整性设计思想：局部、稳定的规则用 `CHECK`、`UNIQUE` 和外键表达；跨表和上下文相关规则用触发器表达；用户体验相关的提示和权限入口仍由应用层负责。

6.11 本章小结

本章说明了当前系统已经从“约束设计方案”推进到“数据库层实现”。主键、外键、唯一约束、`CHECK`、`DEFAULT` 保证基础完整性；角色、容量、先修、选课和成绩日志等跨表规则由触发器维护；应用层负责权限入口、事务调用和友好错误提示。尤其需要强调的是，触发器没有恢复 `selected_count`，而是在保留 3NF 结构的前提下承担复杂完整性检查和审计职责。

第 7 章 事务管理、锁机制与并发控制

7.1 选课系统中的并发一致性问题

选课系统最典型的并发问题是多个学生同时抢同一教学班的最后一个名额。如果容量检查和写入选课记录不在同一事务隔离边界内，两个事务可能同时读到相同的剩余容量，并分别插入选课记录，最终造成超选。

另一个并发问题是同一学生在多个浏览器窗口重复提交选课请求。该问题一方面由数据库函数和触发器检查已有记录，另一方面由 enrollment 表上的 UNIQUE(student_id, offering_id) 在数据库层兜底。

7.2 选课事务边界设计

db/connection.py 中的 DBSession 是项目的事务封装。它进入上下文时建立连接，正常退出时提交事务，发生异常时回滚事务。本轮改造后，学生选课、退课和成绩更新都使用显式事务路径。

学生选课由 services/selection_service.py 调用数据库函数 select_course_tx(p_student_id, p_offering_id)。openGauss 存储函数本身不显式开始或提交事务，而是在调用方事务中执行带锁检查和写入。因此事务边界由 DBSession 管理，数据库函数和触发器在同一事务上下文内工作。

7.3 选课事务流程

当前选课事务流程见表 7-1。其核心原则是：容量检查、时间冲突检查、先修课程检查和写入 enrollment 必须处于同一事务边界内。

表 7-1 选课事务流程

步骤	数据库操作	涉及表	一致性目标	并发控制机制
1	服务层打开 DBSession 并调用 select_course_tx()	无	建立提交或回滚边界	应用层事务封装
2	查询并锁定同一学生同一教学班的已有记录	enrollment	处理重复选课、退课后恢复或已完成课程重选	FORUPDATE
3	锁定目标教学班	course_offering	串行化同一班次容量检查	FORUPDATE
4	插入或更新选课记录	enrollment	形成新的选课关系	触发器随语句执行
5	检查学生、班次和学期状态	student、course_offering、semester	阻止非在读学生或关闭班次选课	触发器兜底

步骤	数据库操作	涉及表	一致性目标	并发控制机制
6	统计当前已选人数	enrollment	防止超过 max_capacity	行锁后聚合统计
7	检查时间冲突	course_schedule、 enrollment	同一学生同学期课程时间 不重叠	同一事务内查询
8	检查先修课程	course_prerequisite、 enrollment	未通过先修课不能选后续 课	同一事务内查询
9	由唯一约束兜底	enrollment	防止重复记录落库	UNIQUE(student_id, offering_id)

关键函数调用形式如下：

```
1 SELECT select_course_tx(:student_id, :offering_id);
```

该设计把业务检查尽量下沉到数据库层，应用层主要负责开启事务和把数据库异常转换为用户可理解的提示。

7.4 行级锁防止并发超选

防止超选的关键是对目标 course_offering 行加行级锁。数据库函数和 trg_enrollment_guard 都使用 SELECT...FORUPDATE 锁定目标教学班：

```
1 SELECT course_id, semester_id, max_capacity, status
2 FROM course_offering
3 WHERE offering_id = :offering_id
4 FOR UPDATE;
```

当多个事务同时选择同一个教学班时，后到事务必须等待先到事务提交或回滚。已选人数统计发生在获得该行锁之后：

```
1 SELECT COUNT(*)
2 FROM enrollment
3 WHERE offering_id = :offering_id
4 AND status = 'selected';
```

由于同一教学班的容量检查被串行化，两个事务不会同时基于同一个旧容量状态插入记录，因此可以防止“最后一个名额被多人同时抢到”的超选问题。系统没有恢复 selected_count 字段，而是通过锁加聚合统计实现一致性。

7.5 唯一约束与重复选课防护

select_course_tx() 会先查询并锁定同一学生、同一教学班的已有 enrollment 记录。如果记录已经是 selected，函数直接拒绝；如果记录已经 completed，也拒绝重复选择；如果是 dropped，则尝试恢复为 selected，并由触发器重新执行容量、

时间冲突和先修课程检查。

即使应用层或函数逻辑遗漏，`UNIQUE(student_id, offering_id)` 仍会在数据库层拒绝重复记录。这是用唯一约束维护业务候选键的典型做法。

7.6 时间冲突与先修课程检查的事务一致性

时间冲突和先修课程检查也放在 `trg_enrollment_guard` 中执行。由于触发器与触发它的 `INSERT` 或 `UPDATE` 语句处在同一事务内，如果检查失败，选课写入会一起回滚。时间冲突使用 `course_schedule` 的结构化时间片判断区间重叠；先修课程使用 `course_prerequisite` 和历史 `completed` 选课记录判断是否达到 60 分。

7.7 成绩更新事务与审计一致性

成绩更新也已经改为事务化。服务层在同一 `DBSession` 中锁定目标 `enrollment` 行，检查教师或管理员权限，然后设置审计上下文：

```
1 SELECT set_config('app.current_user_id', :user_id, true);
2 SELECT set_config('app.score_change_reason', :reason, true);
3 UPDATE enrollment
4 SET final_score = :score, status = 'completed'
5 WHERE enrollment_id = :enrollment_id;
```

`trg_score_change_log` 在 `final_score` 更新后自动写入 `score_change_log`。如果成绩更新失败，日志不会产生；如果日志写入失败，成绩更新也会回滚。因此成绩和审计日志处于同一事务一致性边界内。

7.8 事务隔离级别分析

触发器不是独立事务，而是在触发它的 `INSERT` 或 `UPDATE` 语句所在事务中执行。也就是说，`trg_enrollment_guard` 抛出异常会使选课语句失败并回滚，`trg_score_change_log` 插入日志失败也会使成绩更新失败。这一机制体现了事务的原子性和一致性。

在本系统中，触发器负责数据库层兜底，服务层负责事务入口、用户权限和友好错误提示。二者配合后，选课容量、时间冲突、先修课程、退课限制和成绩日志不再只是前端或服务层约定，而成为数据库可以强制执行的规则。

当前系统没有显式调整全局事务隔离级别，而是针对高风险资源使用行级锁。对选课容量而言，锁定 `course_offering` 行已经能把同一教学班的容量检查串行化；对重复选课而言，业务唯一约束可以在较低隔离级别下兜底；对成绩更新而言，服务层锁定目标 `enrollment` 行后再更新成绩，避免同一成绩记录并发覆盖。

7.9 当前实现的不足与改进

表 7-2 汇总了当前系统面向典型并发场景的处理方式。

表 7-2 并发风险与解决机制

并发风险	可能后果	数据库层解决机制	应用层配合
多个学生同时抢最后一个名额	超过 max_capacity	锁定 course_offering 行后统计 enrollment	DBSession 调用 select_course_tx()
同一学生多窗口重复提交	出现重复选课记录	select_course_tx() 锁定既有记录, UNIQUE(student_id, offering_id) 兜底	把数据库异常转换为友好提示
教师更新成绩时需要审计	成绩变更无追踪或日志与成绩不一致	trg_score_change_log 与成绩更新同事务执行	同事务设置 app.current_user_id 和修改原因
退课与成绩录入并发	已录入成绩的课程被退课	退课锁定目标 enrollment, 触发器禁止有成绩或已完成记录退课	退课接口使用 DBSession
管理员修改教学班容量时学生正在选课	容量降低造成已选人数超过上限	trg_offering_capacity_check 检查容量不能低于当前已选人数	管理页面限制容量输入下限

当前已处理最核心的抢课容量风险，但仍有改进空间：同一学生同一学期选择同一课程不同教学班的限制尚未固化，需要跨表触发器或额外业务键设计；此外，还应继续编写多连接并发压测，模拟多个学生同时抢同一教学班最后名额，验证锁等待和异常回滚行为。

7.10 本章小结

本章从事务和锁角度说明了当前系统如何处理选课一致性。系统没有依赖页面端“先查容量再插入”的脆弱流程，而是在 DBSession 事务中调用 select_course_tx()，由行级锁、触发器和唯一约束共同保证选课写入的一致性。成绩更新同样通过事务上下文和触发器保证成绩与审计日志原子一致。

第 8 章 物理结构设计与性能优化

8.1 数据库系统选择

当前项目使用 openGauss 数据库，并通过 Docker 在本地运行。Docker Compose 文件位于 `opengauss_setup/docker/docker-compose.yml`，容器端口映射为宿主机 15432 到容器 5432。Python 端通过 `psycopg2` 连接，配置见 `config.py`。

选择 openGauss 的意义在于，它是面向企业级场景的关系数据库系统，支持标准关系模型、事务、锁、索引、约束等数据库课程核心内容，适合用于本课程大作业展示。

8.2 数据类型设计

业务编号如 `student_id`、`teacher_id`、`course_id` 使用 `VARCHAR`，便于保存带前缀或学号格式的业务编码。代理主键如 `user_id`、`offering_id`、`enrollment_id` 使用 `BIGSERIAL`，便于自动生成。成绩和学分使用 `DECIMAL`，避免浮点误差。日期和时间使用 `DATE` 与 `TIMESTAMP`，可支持学期范围和选课窗口判断。

8.3 索引设计

主键和唯一约束会自动生成索引。此外，当前 `init.sql` 显式创建了以下常用索引：

- `idx_user_session_user` 与 `idx_user_session_valid`：支持持久登录会话校验。
- `idx_offering_course_semester`：支持按课程和学期查询开课班次。
- `idx_offering_teacher_semester`：支持教师按学期查看负责班次。
- `idx_schedule_offering` 与 `idx_schedule_time`：支持上课时间片展示和时间冲突检查。
- `idx_enrollment_student_status` 与 `idx_enrollment_offering_status`：支持学生成绩单、已选课程、班次人数统计和容量检查。
- `idx_score_log_enrollment` 与 `idx_score_log_changed_at`：支持按选课记录追踪成绩变更，以及按时间倒序查看审计日志。
- `idx_course_prereq_prereq`：支持从某门先修课反查后续课程，并辅助先修关系递归检查。

8.4 规范化后的索引补偿

规范化设计提高了数据一致性，但也会增加表连接数量。例如，课程列表需要从 `course_offering` 连接 `course`、`teacher`、`semester`、`classroom`，上课时间需要从 `course_schedule` 聚合得到，已选人数需要从 `enrollment` 统计得到。因此，物理设计需要通过索引补偿规范化带来的查询成本。

`idx_offering_course_semester` 支持按课程和学期查询教学班，适合管理员维护某门课程的开课情况；`idx_offering_teacher_semester` 支持教师查看指定学期教学任务；`idx_schedule_offering` 支持从教学班快速读取上课时间片；`idx_schedule_time` 支持按星期和节次进行时间冲突检测；`idx_enrollment_student_status` 支持学生查看自己已选、已完成或已退课程；`idx_enrollment_offering_status` 支持统计某教学班当前选课人数，也是容量检查的关键索引。

这组索引与第 5 章的规范化改造形成配合：系统不再在基础表中保存 `schedule_text`、`selected_count` 和 `gpa_point`，而是在查询时通过聚合、连接或 CASE 表达式得到展示值。索引设计的作用就是让这种“减少冗余、查询计算”的模式在数据量增长后仍具有可接受的性能。

8.5 视图设计与派生数据查询

本轮在 `migrate_triggers_constraints_20260608.sql` 中新增了三个查询视图，用于在不破坏 3NF 的前提下提高查询便利性。这些视图不是基础表，不承担数据持久化职责，因此不会把 `selected_count` 或展示用 `schedule_text` 重新引入基础关系模式。

表 8-1 规范化后的查询视图

视图	主要字段	设计目的
<code>v_offering_select</code> <code>ed_count</code>	<code>offering_id</code> 、 <code>max_capacity</code> 、 <code>selected_count</code> 、 <code>remaining</code> <code>_capacity</code>	通过聚合 <code>enrollment</code> 统计已选人数和剩余容量，替代基础表中的派生字段。
<code>v_course_offering</code> <code>_detail</code>	课程、教师、学期、教室、容 量、已选人数、剩余容量、展 示用 <code>schedule_text</code>	服务课程查询、教师班次和管理员统计页面，减少应用层重复 JOIN。
<code>v_student_timetab</code> <code>le</code>	学生、课程、教师、教室、星 期、开始节次、结束节次	支持学生课表查询和时间冲突规则说明。

需要强调的是，视图中的 `selected_count` 和 `schedule_text` 是查询结果，不是

基础表字段。它们不会引入基础表更新异常，也不需要触发器维护。视图把规范化结构转换为页面更容易使用的展示结构，体现了物理设计对逻辑设计的补偿。

8.6 高频查询

高频查询主要包括：学生查看可选课程、学生查看已选课程、教师查看某班次学生名单、管理员查看班次容量、成绩单查询、成绩分布统计。当前服务层已经在课程列表、学生可选课程和管理员班次统计中使用 `v_course_offering_detail`；成绩单中的 `gpa_point` 仍通过 CASE 表达式按 `final_score` 动态计算，避免保存派生字段。

8.7 性能风险与优化

当前系统数据量较小，查询性能足以支撑课程演示。但随着数据规模扩大，动态聚合人数和时间冲突查询可能成为热点。可继续优化的方向包括：

- 使用执行计划分析高频 SQL，确认索引是否被使用。
- 对成绩分布、班次容量等统计页面增加分页或限制。
- 对复杂报表考虑物化视图或定期统计表，但需权衡规范化和一致性。
- 对模糊查询字段增加合适索引或限制查询范围。

8.8 备份恢复

当前项目提供初始化脚本 `opengauss_setup/docker/init_db.sh`，可以重建数据库并导入样例数据。项目还提供 `migrate_3nf_20260608.sql`，用于旧结构迁移到 3NF 结构。正式运行环境中应增加定期备份、恢复演练和迁移版本管理，避免依赖手工操作。

第 9 章 系统运行、维护与数据库管理

9.1 数据库初始化与部署

openGauss 本地部署由 `opengauss_setup/docker/start_opengauss.sh` 和 `init_db.sh` 完成。前者启动 Docker 容器并调用初始化脚本，后者负责等待数据库可连接、重建 `course_system` 数据库、导入 `opengauss_setup/sql/init.sql`，并继续执行 `migrate_triggers_constraints_20260608.sql` 以安装约束、触发器、函数和视图。

这种方式便于课程演示和本地复现，但正式环境应避免随意重建数据库，建议使用迁移脚本和备份策略管理结构变更。

9.2 迁移脚本与部署顺序

当前数据库结构由三个关键脚本体现演进过程：`init.sql` 定义基础表、主键、外键、样例数据和基础索引；`migrate_3nf_20260608.sql` 将旧结构迁移到 3NF，包括拆分 `course_schedule` 并删除 `schedule_text`、`selected_count` 和 `gpa_point`；`migrate_triggers_constraints_20260608.sql` 在 3NF 结构基础上补充业务唯一约束、CHECK、触发器、存储函数、视图和审计索引。

部署顺序需要遵循“基础表与数据先就绪，再安装复杂约束和触发器”的原则。原因是触发器会主动检查容量、角色和选课窗口，如果在样例数据修正前启用，可能导致初始化数据导入失败。当前迁移脚本在添加容量触发器前会修正已有样例数据中教学班容量超过教室容量的问题。

9.3 测试脚本与验证

`opengauss_setup/sql/test_triggers_constraints.sql` 是本轮数据库层规则的回滚型验证脚本。它在一个事务中构造测试数据，覆盖角色错配、先修自指、先修环路、容量超选、时间冲突、重复选课、成绩审计、退课限制和视图查询，最后执行 `ROLLBACK`，不会污染样例数据。该脚本的存在使触发器和约束不仅停留在设计说明中，而是可以被重复验证。

9.4 权限管理

应用层权限主要通过 `user_account.role`、页面守卫 `require_role()` 和服务层权限检查实现。教师成绩管理中，`score_service.can_manage_offering_score()` 会检查教师只能维护本人负责的开课班次；管理员具有全局维护能力。

当前项目尚未实现数据库级角色授权，例如为学生、教师、管理员分别建立数据

库账号并授予最小权限。报告中应如实说明：当前权限主要是应用层权限，数据库层权限管理是后续改进方向。

9.5 日志审计

成绩修改日志由 `score_change_log` 表保存，记录 `enrollment_id`、旧成绩、新成绩、修改人、修改时间和原因。该设计保留了成绩变更历史，避免成绩被覆盖后无法追溯。本轮已经通过 `trg_score_change_log` 实现数据库触发器自动写日志；服务层在成绩更新事务中设置当前操作者和修改原因，触发器负责写入审计记录。

9.6 异常处理与一致性维护

`DBSession` 对异常进行回滚，是当前项目一致性维护的重要基础。学生和教师创建流程、开课班次创建与更新时间片、选课、退课和成绩更新流程都使用事务。服务层函数通常返回 `(bool, message)`，页面根据返回结果提示用户。

数据库层触发器抛出的异常会被服务层捕获并转换为用户可以理解的提示。例如，`selection_service.py` 将容量已满、时间冲突、先修课程未完成等数据库异常映射为中文提示；`score_service.py` 将缺少 `app.current_user_id` 的成绩更新异常映射为审计上下文缺失提示。仍需补充的异常处理包括：数据库连接失败提示、迁移失败回滚策略、删除课程或学生时历史记录的保留策略等。

9.7 视图与查询维护

`v_offering_selected_count`、`v_course_offering_detail` 和 `v_student timetable` 把多表连接和聚合统计封装在数据库层，降低了应用层重复编写 JOIN 的成本。由于视图不存储派生数据，它们不会破坏 3NF；同时，底层索引支持容量统计、课程列表和课表查询，便于后续维护和性能调优。

9.8 数据安全

项目中密码以 SHA-256 哈希保存，持久登录 token 以哈希形式存入 `user_session`。这比明文保存更安全，但仍可继续改进，例如使用带盐的密码哈希算法、限制登录尝试次数、设置 HTTPS、定期清理过期会话。

9.9 后续扩展

后续可扩展方向包括：行政班实体拆分、绩点规则版本化、数据库用户权限、备份恢复脚本、运行监控、并发压测和报表优化。这些内容可以作为运行维护阶段的设计完善点。

第 10 章 总结与不足

10.1 系统设计总结

本项目围绕高校选课业务建立了较完整的关系数据库模型。系统区分课程定义和开课班次，使用 `enrollment` 转换学生与教学班之间的多对多联系，使用 `course_schedule` 结构化表达上课时间片，并通过 `course_prerequisite` 表示课程先修自关联。当前数据库结构由 `opengauss_setup/sql/init.sql` 和后续迁移脚本维护，共包含 15 张主要表。

10.2 课程知识点体现

项目较好体现了数据库原理 2 的多个重点内容：

1. 数据库设计流程：报告按规划、需求、概念、逻辑、规范化、物理和运行维护展开。
2. ER 建模：实体、属性、联系和基数均能从真实表结构中抽取。
3. 关系模型：M:N 联系通过 `enrollment` 和 `course_prerequisite` 转换为关系模式。
4. 规范化：通过删除 `schedule_text`、`selected_count`、`gpa_point` 等冗余字段，核心表更接近 3NF。
5. 完整性：主键、外键、唯一约束、CHECK、DEFAULT 和触发器共同维护数据正确性。
6. 事务与锁：选课流程使用 `select_course_tx()`、`DBSession` 和 `SELECT...FOR UPDATE` 控制并发。
7. 物理设计：项目为会话、教学班、时间片、选课记录和成绩日志建立了常用索引，并用视图提升查询便利性。
8. 测试验证：`test_triggers_constraints.sql` 覆盖容量、时间冲突、重复选课、先修关系、成绩审计和视图查询等数据库规则。

10.3 项目不足

当前不足主要包括：同课跨班重复选择限制尚未完全固化；数据库级权限、备份恢复、运行监控和多连接并发压测还不完善；当前使用普通视图而非物化视图，统计报表在数据量很大时仍需进一步评估性能。

10.4 后续改进

如果继续完善项目，优先级较高的数据库相关功能包括：

1. 为同课跨班重复选择设计跨表触发器或培养方案规则表。
2. 增加多连接并发压测，验证最后一个名额抢课时的锁等待和回滚行为。
3. 建立更完整的数据库管理方案，包括数据库级角色权限、备份恢复和迁移版本管理。

总体而言，本项目已经不是简单的页面 CRUD 程序，而是能够体现数据库设计流程、范式理论、完整性约束、事务锁机制和运行维护思路的数据库应用系统。报告中的不足部分也说明数据库设计需要随着业务规则和运行环境持续迭代。

附录 A 关键 SQL 与测试摘录

A.1 选课事务函数摘录

本系统通过 `select_course_tx()` 封装数据库层选课入口。函数在调用方事务中执行，先锁定同一学生同一教学班的既有选课记录，再锁定目标教学班，最终由 `trg_enrollment_guard` 完成完整性兜底。

```
1 CREATE OR REPLACE FUNCTION select_course_tx(  
2     p_student_id VARCHAR,  
3     p_offering_id BIGINT  
4 ) RETURNS BIGINT AS $$  
5 ...  
6     PERFORM 1  
7     FROM course_offering  
8     WHERE offering_id = p_offering_id  
9     FOR UPDATE;  
10  
11     INSERT INTO enrollment (student_id, offering_id, status, select_time)  
12     VALUES (p_student_id, p_offering_id, 'selected', CURRENT_TIMESTAMP)  
13     RETURNING enrollment_id INTO v_enrollment_id;  
14 ...  
15 $$ LANGUAGE plpgsql;
```

A.2 角色一致性触发器摘录

角色档案一致性由通用函数和三个触发器共同实现。下面片段说明普通外键之外，系统还会检查账户角色是否与档案表类型匹配。

```
1 CREATE OR REPLACE FUNCTION check_user_role_fn(  
2     p_user_id BIGINT,  
3     p_expected_role VARCHAR  
4 ) RETURNS void AS $$  
5 ...  
6     IF v_role <> p_expected_role THEN  
7         RAISE EXCEPTION 'User % role is %, expected %',  
8             p_user_id, v_role, p_expected_role;  
9     END IF;  
10 END;
```

```

11 $$ LANGUAGE plpgsql;
12
13 CREATE TRIGGER trg_student_role_check
14 BEFORE INSERT OR UPDATE OF user_id ON student
15 FOR EACH ROW
16 EXECUTE PROCEDURE trg_profile_role_check_fn('student');

```

A.3 选课完整性触发器摘录

trg_enrollment_guard 不维护 selected_count，而是在锁定教学班后统计 enrollment 中的 selected 记录数。

```

1 SELECT course_id, semester_id, max_capacity, status
2 INTO v_course_id, v_semester_id, v_max_capacity, v_offering_status
3 FROM course_offering
4 WHERE offering_id = NEW.offering_id
5 FOR UPDATE;
6
7 SELECT COUNT(*) INTO v_selected_count
8 FROM enrollment e
9 WHERE e.offering_id = NEW.offering_id
10 AND e.status = 'selected'
11 AND (v_old_enrollment_id IS NULL OR e.enrollment_id <> v_old_enrollment_id)
12 ;
13 IF v_selected_count >= v_max_capacity THEN
14 RAISE EXCEPTION 'Course offering % is full', NEW.offering_id;
15 END IF;

```

A.4 成绩审计触发器摘录

成绩审计触发器要求业务层在同一事务中传入当前操作者，避免把日志操作者写死为管理员或空值。

```

1 BEGIN
2     v_changed_by_user_id := current_setting('app.current_user_id')::BIGINT;
3 EXCEPTION WHEN OTHERS THEN
4     RAISE EXCEPTION
5         'Score update requires app.current_user_id in the current transaction';
6 END;
7
8 INSERT INTO score_change_log

```

```

9      (enrollment_id, old_score, new_score, changed_by_user_id, reason)
10  VALUES
11      (NEW.enrollment_id, OLD.final_score, NEW.final_score,
12      v_changed_by_user_id, v_reason);

```

A.5 视图 SQL 摘录

v_offering_selected_count 通过聚合实时得到已选人数和剩余容量，替代基础表中的派生字段 selected_count。

```

1  CREATE OR REPLACE VIEW v_offering_selected_count AS
2  SELECT
3      co.offering_id,
4      co.max_capacity,
5      COALESCE(COUNT(e.enrollment_id), 0)::INTEGER AS selected_count,
6      (co.max_capacity - COALESCE(COUNT(e.enrollment_id), 0))::INTEGER
7      AS remaining_capacity
8  FROM course_offering co
9  LEFT JOIN enrollment e
10     ON e.offering_id = co.offering_id
11     AND e.status = 'selected'
12  GROUP BY co.offering_id, co.max_capacity;

```

A.6 SQL 测试结果摘录

opengauss_setup/sql/test_triggers_constraints.sql 使用事务和匿名块捕获预期异常，最后执行 ROLLBACK，不会污染样例数据。测试覆盖角色错配、先修自指、先修环路、容量超选、时间冲突、重复选课、成绩审计、退课限制和视图查询。一次测试输出摘录如下：

```

1  PASS role check teacher-as-student failed
2  PASS self prerequisite failed
3  PASS prerequisite cycle failed
4  PASS capacity full failed
5  PASS timetable conflict failed
6  PASS unique student/offering failed
7  PASS score update without user failed
8  PASS score audit trigger inserted log row
9  PASS completed enrollment drop failed
10 Trigger and constraint tests finished with rollback.

```

参考资料

1. 项目仓库文件: `opengauss_setup/sql/init.sql`、`services/*.py`、`db/connection.py`、`README.md`。
2. 数据库原理 2 课堂笔记: 数据库设计流程、范式、事务、并发控制、完整性、安全性相关章节。
3. openGauss 与 PostgreSQL 兼容 SQL 特性文档, 用于理解主键、外键、CHECK、索引和 `SELECT...FORUPDATE`。